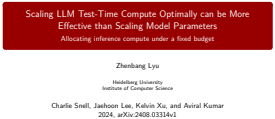
Test-Time Compute in LLMs


Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

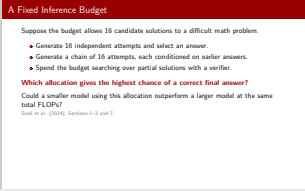
Allocating inference compute under a fixed budget

Zhenbang Lyu

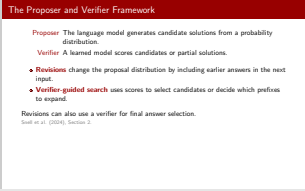
Heidelberg University
Institute of Computer Science

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar
2024, arXiv:2408.03314v1

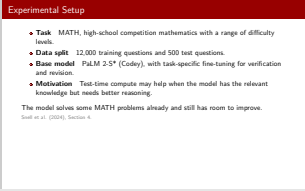
Time 0:00–0:30. The paper asks how to improve a fixed model by spending more computation when it answers a question. We will first examine how to allocate that computation, then compare it with training a larger model.



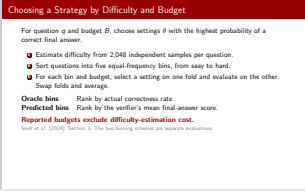
Time 0:30–1:30. Define test-time compute as computation after training, used to answer a question. Keep the same problem and same budget while changing how we spend it. The 16 candidates are an illustrative budget. The second question will require counting training and inference together.



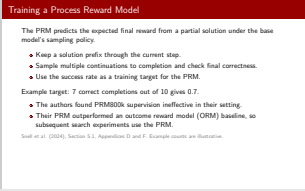
Time 1:30–2:45. Distribution means the probabilities of different generated responses. Feeding back an earlier answer changes the conditioning input and therefore these probabilities. A verifier supplies scores. Clarify that the two routes can share a final selection mechanism. Their distinction concerns how candidates are generated.



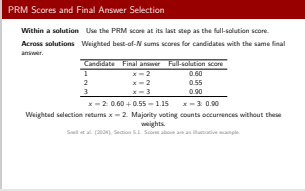
Time 2:45–3:45. These are experiments with a particular model and task. The revision and verification capabilities are trained. The paper studies a regime where basic knowledge is present and better use of computation might recover correct solutions.



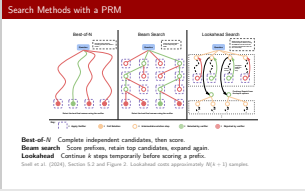
Time 3:45–5:30. The exact optimum is unknown for a new question. Difficulty is a proxy for selecting settings. Oracle and predicted scores produce two separate partitions. Predicted scores permit ranking without ground-truth labels, but are not actual success rates. Two-fold validation separates selecting a strategy from measuring its result. This is optimal among tested settings. The 2,048-sample difficulty cost is excluded from reported inference budgets.



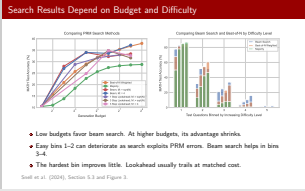
Time 5:30–7:00. Monte Carlo rollouts create supervision before the search experiments. A label of 0.7 estimates future success from the whole prefix under this base model. The trained PRM can then predict such values directly. ORM scores complete outcomes. The paper reports that their PRM performs better. Mention that the authors suspect distribution shift between GPT-4 solutions in PRM800k and PaLM 2 outputs.



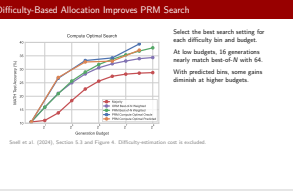
Time 7:00–8:15. Distinguish training a scorer from deciding how to use its scores. The last-step score uses the entire solution prefix. The aggregate 1.15 is a sum of scores, not a probability. Standard best-of-N would choose the single highest-scoring candidate here. Weighted best-of-N pools support for the same final answer. Majority also groups identical answers, but gives every candidate equal weight.



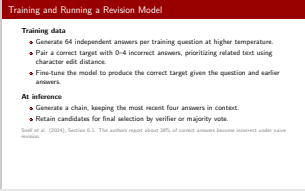
Time 8:15–10:00. Use the diagram left to right. With N=6 and M=2, the paper's beam procedure scores six prefixes, retains three, and extends each twice. Lookahead keeps this process but evaluates each prefix using a temporary greedy continuation of up to k steps. k=0 is ordinary beam search. Its extra simulation costs count toward the budget. All methods eventually use final candidate selection.



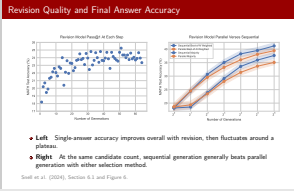
Time 10:00–12:00. Read the left graph first: x is generation budget, y is MATH accuracy. Stronger search need not produce better answers because it optimizes an imperfect scorer. Then read the right graph: each group is a difficulty bin and bars reflect increasing budgets. The medium bins benefit, while easy bins can degrade. The authors observed repetitive low-information steps and overly short answers in search outputs. This motivates choosing the strategy conditionally.



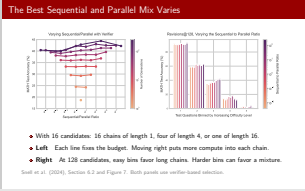
Time 12:00–13:00. Connect this back to the policy in Section 3. Separate oracle from predicted difficulty. The low-budget result is about approaching best-of-N using one-quarter of the generation budget. Do not generalize it into a fourfold saving at every high budget. Search and revisions have different results and examples.



Time 13:00–14:15. The 64 training answers are independent and are assembled into synthetic trajectories afterward. Edit distance is a training-data selection heuristic. During inference, only the most recent four responses are fed back, while the full candidate sequence is stored. Correct answers can be revised into wrong ones because training context contained incorrect answers. Final selection addresses this risk.

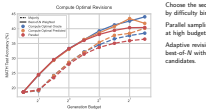


Time 14:15–16:00. The left plot averages correctness at each fixed position across four 64-answer trajectories per test question, then across questions. A 64-answer chain is compatible with a four-answer context window. The right plot measures accuracy after selecting among N candidates. Its four curves combine sequential/parallel generation with weighted verifier selection/majority vote. It does not show the baseline of always submitting the final revision.



Time 16:00–17:45. Define horizontal ratio as chain length divided by number of independent chains. Their product is total candidate count. For the combined strategy, the verifier first selects within each chain, then across chains, as shown in Figure 5. In the left plot, line color encodes budget. In the right plot, color instead encodes sequential/parallel ratio. Right-panel difficulty rises from bins 1 to 5. Independent starts give different overall approaches, while revisions refine an existing approach. The hardest bin remains low across settings.

Compute-Optimal Revisions Improve Scaling



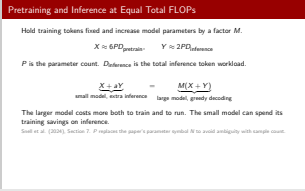
Choose the sequential/parallel ratio by difficulty bin and budget.

Parallel sampling begins to plateau at high budgets.

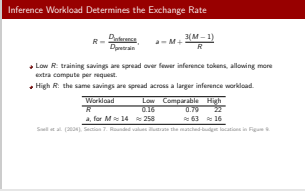
Adaptive revisions can beat parallel best-of-N with 64 rather than 256 candidates.

Snell et al. (2024), Section 6.2 and Figure 8. Difficulty-estimation cost is excluded.

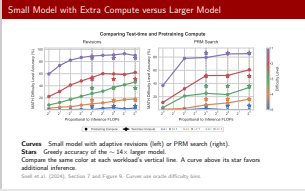
Time 17:45–18:45. Solid curves use weighted best-of-N, dashed curves use majority vote. Colors distinguish oracle policy, predicted policy, and pure parallel generation. Both difficulty definitions improve scaling. This is the revisions-specific 64-versus-256 example. The fourfold claim concerns generation budgets in these experiments.



Time 18:45–20:15. We now compare different model sizes and count training plus inference. P denotes parameters to keep it distinct from earlier candidate count N. X and Y are the small model's baseline costs. The coefficients approximate forward compute and forward-plus-backward compute. More parameters increase both costs. The paper compares parameter scaling while keeping pretraining token count fixed.



Time 20:15–21:30. Solve X+aY=M(X+Y), then substitute X/Y=3/R. a multiplies the small model's baseline inference compute. The larger model's accuracy stays the same as R changes. What changes is how much inference budget the small model gets at equal total cost. R describes total expected workload, rather than the length of one answer.

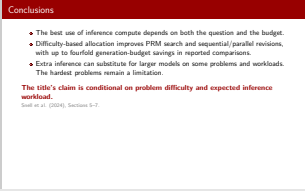


Time 21:30–23:45. Difficulty colors match curves and stars. Stars repeat at three x positions, with the same y value, because matched compute differs by workload. For the left green curve, at low matched budget the star is higher, while at high matched budget the curve is higher. Easy/intermediate problems and low inference workload often favor extra inference. Hard problems and high workload often favor more pretraining. The result uses oracle difficulty and fixed training token counts.

Limits of the Comparison
- Estimating difficulty needs substantial compute, excluded from reported budgets.
- Hardest problems gain little from the tested search and revision methods.
- The evidence comes from PaLM 2-S* on MATH, with capability-specific fine-tuning.
- The FLOPs comparison scales parameters at fixed training data and uses oracle difficulty bins.

Snell et al. (2024), Sections 3–4, 7–8.

Time 23:45–24:30. Real deployment must pay for difficulty estimation. The data and model constrain generality. The larger-model comparison is one parameter-scaling setting. The paper leaves the combination of PRM tree search with revision chains for future study.



Time 24:30–25:00. Return to the opening question. A fixed generation budget can be spent in very different ways. Choosing adaptively improves efficiency. The larger-model claim follows only under certain problem and workload conditions. End here at 25 minutes, excluding questions.

Thank you

Questions and Discussion

Questions begin after the 25-minute talk. Backup slides contain details on validation, rollouts, and FLOPs accounting.

Backup: Difficulty Bins and Cross-Validation

- Score every question using 2,048 samples. Sort and divide into five quantiles.
- Within a bin, split questions into folds A and B.
- Use fold A to select settings at each budget. Evaluate those settings on B.
- Swap A and B. Average held-out performance.

Oracle and predicted difficulty repeat this procedure with different scoring rules.

Snell et al. (2024), Section 3.2.

Optional backup. Oracle uses actual correctness, predicted difficulty uses mean verifier scores. The bin boundaries are quantiles of scores across questions. A score such as 0.7 has no universal fixed bin.

Backup: Training Rollouts and Lookahead

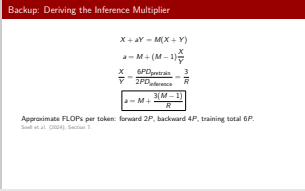
PRM training Sample multiple continuations from a prefix to estimate expected final correctness. This supplies supervision.

Lookahead search At inference time, generate up to k extra steps with temperature 0. Use the trained PRM's score at that endpoint to evaluate the current prefix.

The temporary lookahead steps help choose which current prefixes to retain. Beam search is the $k = 0$ case.

Snell et al. (2024), Sections 5.1–5.2.

Optional backup. These two uses of rollout occur at different stages. Training rollouts create target values, while lookahead consults a trained PRM during search and incurs extra inference cost.



Optional backup. Backward computation forms gradients for weights and for activations, with each major matrix computation comparable to a forward pass. Total P includes parameters in every layer, so layer count is already accounted for. These approximations omit some operations.